

TO SAVE THE BEST MODEL



joblib.dump (best_pipeline, "mental_health_pipeline.pkl")

BUILDING FAST-API
PYDANTIC BACKEND

- ① import joblib
- ② LOAD THE MODEL

↳ model = joblib.load ("Mental_health_model.pkl")

③ TO RUN A FAST-API SERVER

```
python -m unicorn main:app --reload.
```

④ CREATE A POST ROUTE

```
@app.post('/predict')  
def predict():
```

⑤ CREATE A PYDANTIC MODEL

```
class StudentData(BaseModel):  
    age: int  
    gender: str  
    :  
    :  
    stress_level: str
```

⑥ USE THE ABOVE MODEL AS THE REQ BODY OF /predict route.

⑦ `prediction = model.predict(input_row)[0]`

DEEP LEARNING

When traditional learning ends, Deep Learning begins turning raw data into intelligence, and intelligence into discoveries.

What is Deep Learning?

Imagine your brain has lots of tiny switches (neurons). When you see a cat, your brain looks at:

- the shape of the ears.
- The whiskers.
- The fur
- The sound "meow".

Deep Learning works the same way. It builds layers of tiny switches (artificial neurons) inside a computer. Each layer learns something:

- First layer: "This looks like an edge".
- Next layer: "This looks like an ear".
- Final layer: "Oh! It's a rat."

Learning from mistakes until it gets really good.

WHY IS DL POPULAR NOW?

- Data explosion
- Hardware power (GPUs & TPUs)
 - ↳ Training deep networks needs a lot of calculations.

↳ Earlier computers were slow, now we have GPUs & TPUs.

- Automatic feature learning.
- Better performance.
- Versatility Across Domains.
- Research Breakthroughs.

PHASE: 1
FOUNDATIONS

PHASE: 3
TRAINING CHALLENGES

PHASE: 2
CORE MECHANICS

PHASE: 4
CONVOLUTIONAL
NEURAL NETWORKS (CNNs)

PERCEPTRON

The perceptron is the simplest type of artificial neuron. It is the foundation of neural networks and deep learning.

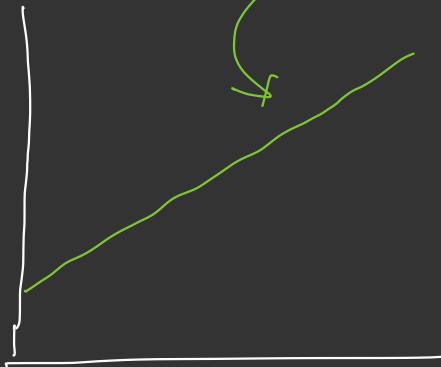
It mimics how a biological neuron works taking inputs, processing them and giving output.

Table $\rightarrow$

Age	cholesterol	BP	heart_disease
28	150	110	1
36	120	90	0
42	150 180	160	1

step-1 $\Rightarrow y = mx + b \rightarrow h_{\theta}(x) = \theta_0 + \theta_1 x_1$

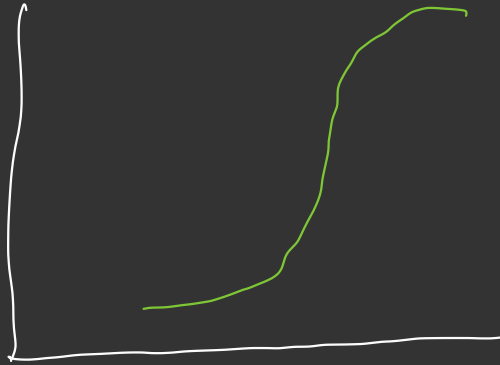
$$h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$



STEP-2

SIGMOID ACTIVATION FUNCTION

$$= \frac{1}{1 + e^{-z}} \Rightarrow z = (\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3)$$

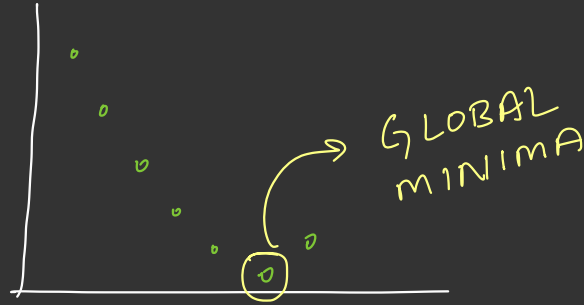


STEP-3

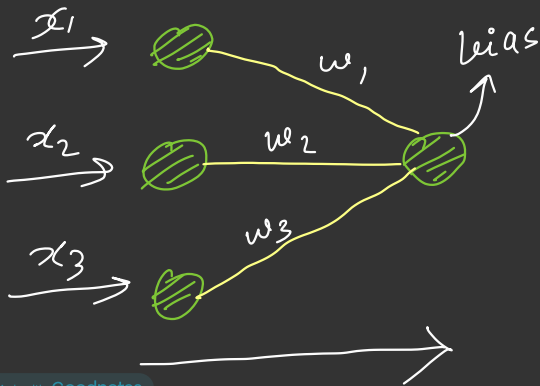
Log loss function
 $\Rightarrow$ loss find.

STEP-4 - OPTIMIZER

$$\theta_0 + \theta, x,$$



we did all of these using PERCEPTRONS.



$$x_1 \times w_1 + x_2 \times w_2 + x_3 \times w_3 + \text{bias}$$

$$(\theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3 + \theta_0)$$

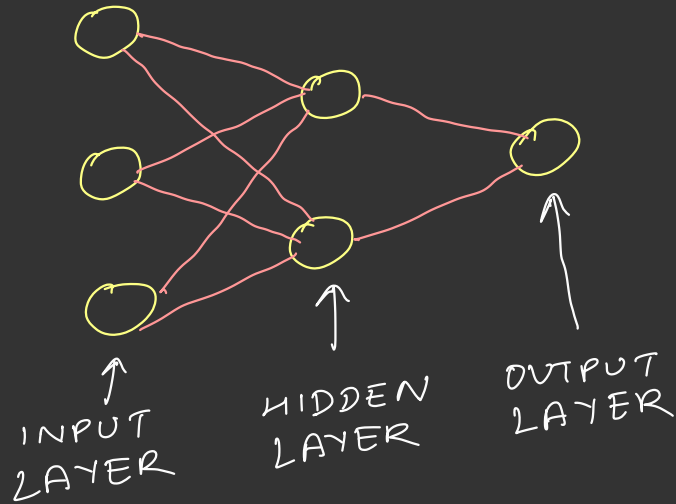
same sigmoid activation function.

$$= \frac{1}{1 + e^{(-2)}}$$

LIMITATIONS OF PERCEPTRONS

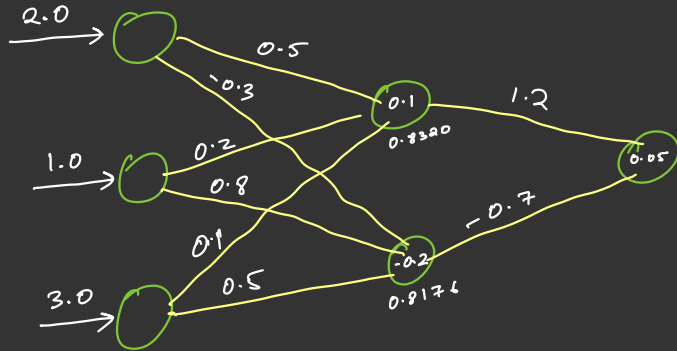
↳ step activation function $\rightarrow 0/1$

ANN (Artificial Neural Network)



FORWARD PROPAGATION

x_1	x_2	x_3	y
2.0	1.0	3.0	1



$$\begin{aligned}
 h_{n_1} &= (x_1 w_1) + (x_2 w_2) + (x_3 w_3) + \text{bias} \\
 &= (2.0)(0.5) + (1.0)(0.2) + (3.0)(0.1) \\
 &\approx 1.6
 \end{aligned}$$

$$\sigma = \frac{1}{1 + e^{-z}}$$

$$\sigma(1.6) = 0.8320$$

$$\text{output neuron} = [0.8320 (1.2)] + [0.8176 (0.7)] + 0.05$$

$$= 0.4761$$

$$\sigma(0.4761) = 0.6168 > 0.5 \\ \approx 1$$

$$\boxed{y_{\text{pred}} = 1}$$

★ 3 IMPORTANT THINGS

ACTIVATION FUNCTION

↓
forward propagation

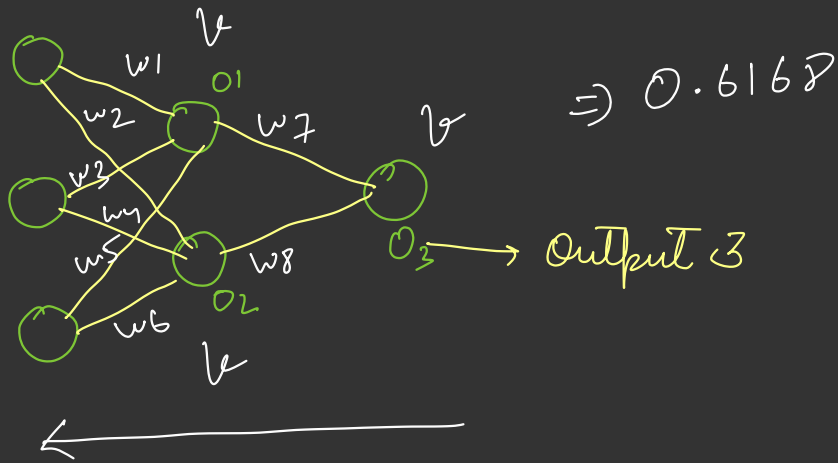
LOSS FUNCTION

OPTIMIZERS

Backward propagation

BACKWARD PROPAGATION

$$\begin{aligned} \hat{y} &= 0.6168 \\ y &= 1 \end{aligned} \rightarrow \begin{aligned} \text{Log loss} &= [y \times \log(\hat{y}) \\ &+ (1-y) \times \log(1-\hat{y})] \\ &= 0.494 \end{aligned}$$



Loss = 0.494 $\downarrow \downarrow$
 we have to lower the Loss.

① UPDATE THE WEIGHTS

$$w_{\text{new}} = w_{\text{old}} - \underbrace{\eta}_{\text{learning rate}} \frac{dL}{dw_{\text{old}}} \quad L = \text{Loss}$$

$$w_{7\text{ new}} = w_{7\text{ old}} - \eta \left[\frac{dL}{dw_{7\text{ old}}} \right] \Rightarrow \frac{dL}{dO_3} \times \frac{dO_3}{dw_{7\text{ old}}}$$

$$w_{1, \text{new}} = w_{1, \text{old}} - \eta \left[\frac{dL}{dw_{1, \text{old}}} \right] \Rightarrow \frac{dL}{dO_3} \times \frac{dO_3}{dO_1} \times \frac{dO_1}{dw_{1, \text{old}}}$$

$$w_{6, \text{new}} = w_{6, \text{old}} - \eta \left(\right) \frac{dL}{dw_{6, \text{old}}} = \frac{dL}{dO_3} \times \frac{dO_3}{dO_2} \times \frac{dO_2}{dO_6}$$

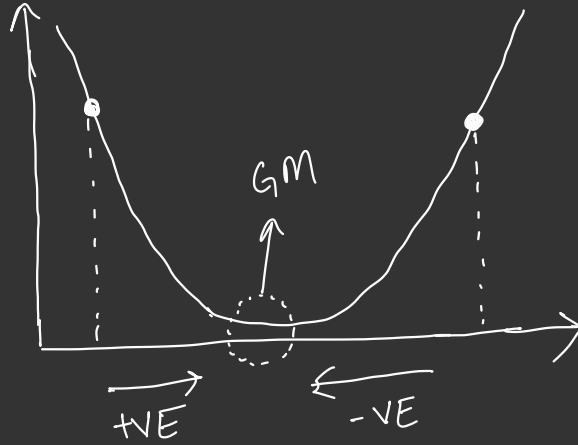


$$w_1, w_2, w_3, w_4, w_5, w_6 \Rightarrow 0.432 \quad \text{LOSS}$$

$$\begin{array}{c} \Downarrow \\ \text{changed} \end{array} \Rightarrow 0.349 \quad \text{LOSS}$$

★ Chain rule of derivatives...

AFTER THIS WE USE OPTIMIZERS LIKE GRADIENT DESCENT



1 epoch = 1 Forward Propagation
+
1 Backward Propagation.

→ 62%

2nd epoch → 69%

⋮

20th epoch → 94%

ACTIVATION FUNCTION ↴

① SIGMOID ACTIVATION FUNCTION

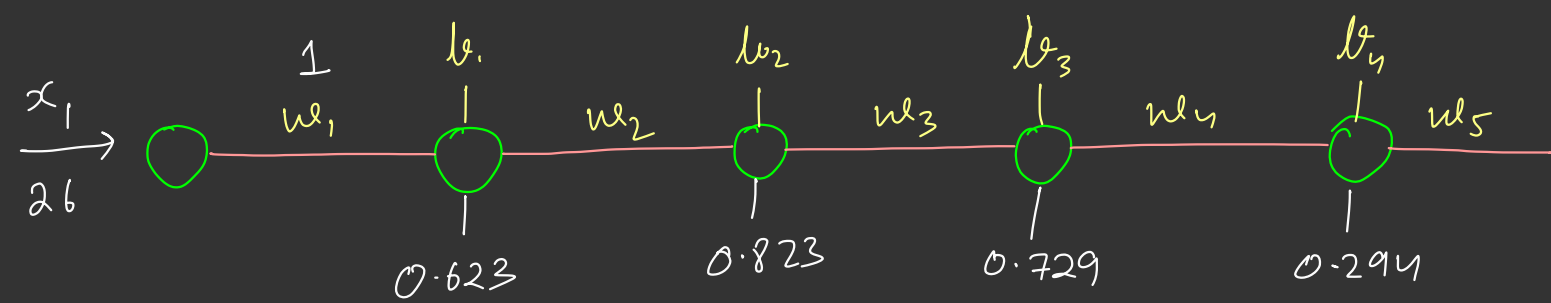
$$\hookrightarrow \sigma = \frac{1}{1 + e^{-z}}$$

A mathematical rule applied to a neuron's weighted input sum. It decides whether the neuron should pass or fire information.

↳ Vanishing gradient problem.

(happens during back propagation.)

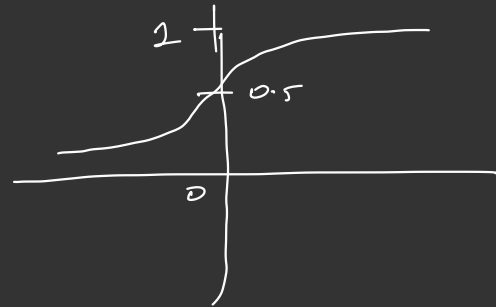
The vanishing gradient problem happens during back propagation when gradients become so tiny that the earlier layers of a deep neural network stop learning.



$$\sigma \left[(26 \times 1) + b \right] = \frac{1}{1 + e^{-z}} \Rightarrow \sigma = \frac{1}{1 + e^{-((26 \times 1) + b)}}$$

$\Downarrow$
 0.623

$$w_{\text{new}} = w_{\text{old}} - \eta \left[\frac{d \text{ Loss}}{d w_{\text{old}}} \right]$$



$$\Rightarrow \frac{dL_{\text{loss}}}{dw_{1,\text{old}}} = \frac{dL_{\text{loss}}}{do_4} \times \frac{do_4}{do_3} \times \frac{do_3}{do_2} \times \frac{do_2}{do_1} \times \frac{do_1}{dw_1}$$

TYPE OF ACTIVATION FUNCTION

① Linear Activation function.

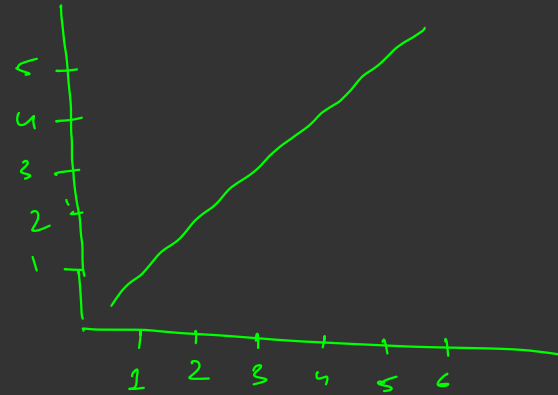
A linear activation function is the simplest type of activation function. It basically means

$$f(x) = x$$

so, the output is the same as the input.

$$[\theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3]$$

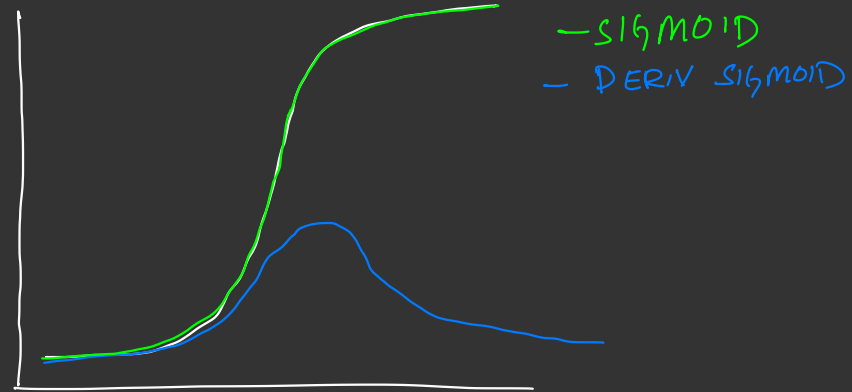
If we use linear activation in all layers, the whole network becomes just a linear model, no matter how many layers add.



② Sigmoid Activation function

The sigmoid function is an S shaped curve that squashes any real number into a range between 0 and 1.

- The limitation is vanishing gradient problem for very large or ~~very~~ very small inputs, the gradient becomes almost 0, which slows learning.



TANH ACTIVATION FUNCTION

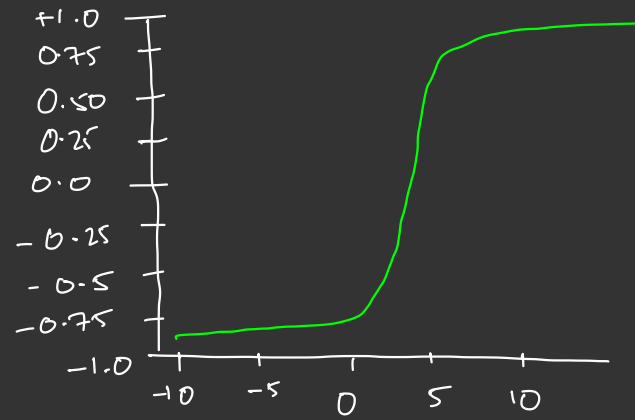
The Tanh function (short for hyperbolic Tangent) is another squashing function like sigmoid, but instead of squeezing values into 0 to 1, it squeezes from -1 to +1.

$$\text{formula : } \tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

where do we use it?

- Often used in hidden layers of neural network.
- Useful when data has both positive and negative values because it centres the output around 0.

- Outputs are zero-centred (good for optimization).
- Stronger gradients than sigmoid in the range $(-1, 1]$, so learning can be faster.



LIMITATIONS

- Still suffer from VANISHING GRADIENT PROBLEM when inputs are very large.

ReLU ACTIVATION FUNCTION

ReLU stands for Rectified Linear Unit.

It's super simple:

$$f(x) = \max(0, x)$$

That means:

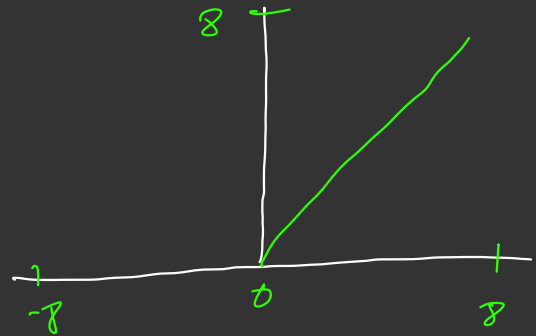
if input $x < 0 \rightarrow \text{output} = 0$

if input $x > 0 \rightarrow \text{output} = x$

So, it either passes positive values as they are or blocks negative values by turning them into 0.

→ Hidden layers of almost all modern deep neural networks

→ Works really well in CNNs (Convolutional neural Net).

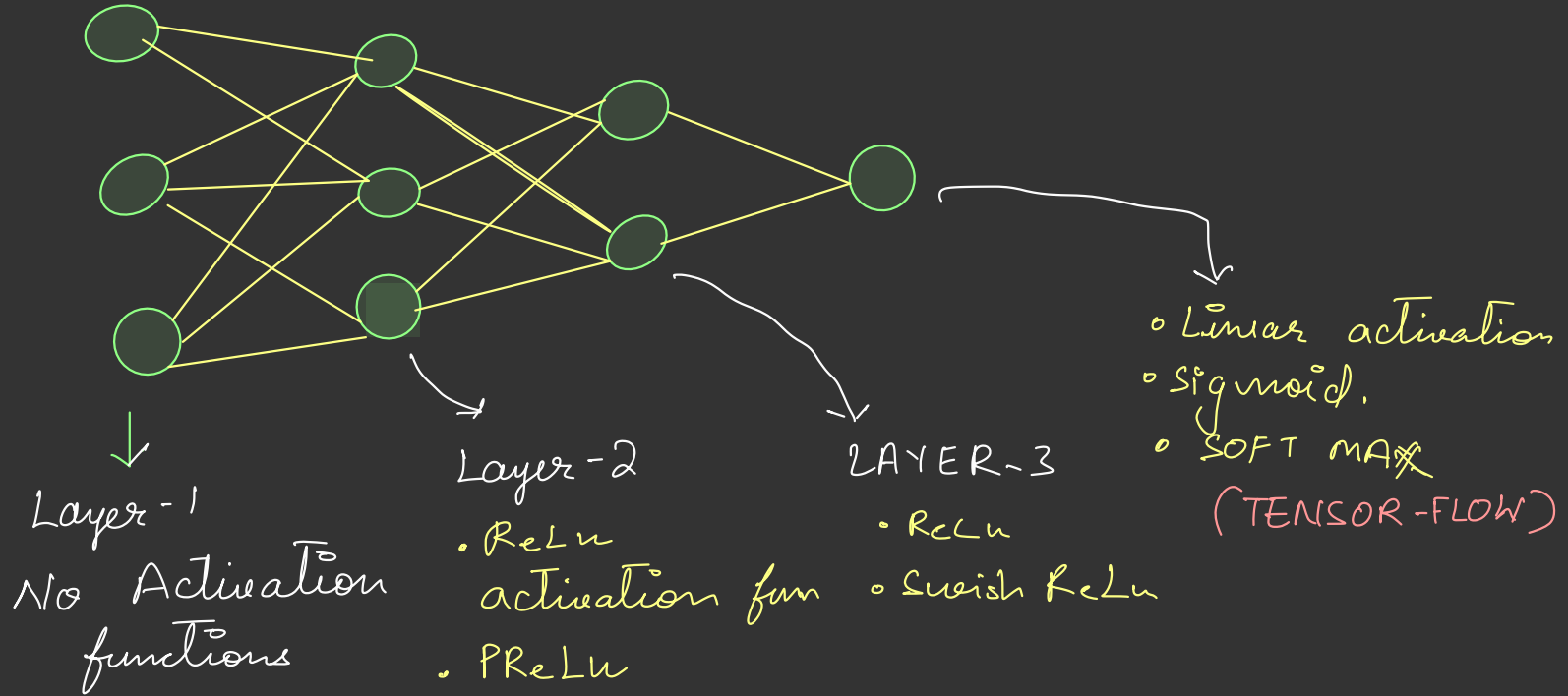


- Very fast and simple to compute.
- Helps avoid vanishing gradient problem
- Makes training deep networks faster.

LIMITATIONS:-

- Dying ReLU problem: sometimes neurons get stuck at 0 forever if weights update badly.
- Not smooth at 0

WHERE TO USE WHICH ACTIVATION FUNCTION ?



We scale the dataset, specifically Min-Max normalization

$$X_{\min} = X.\min()$$

$$X_{\max} = X.\max()$$

$$X_{\text{scaled}} = (X - X_{\min}) / (X_{\max} - X_{\min})$$

So, for each column

- smallest value becomes 0.
- largest value becomes 1.
- middle values are scaled between them.

$$X = df[["", "", "", ""]]$$

$$y = df[""]$$

model = keras.Sequential([
layers.Input(shape = (x_train.shape[1],)) → INPUT
layers.Dense(8, activation = 'relu'), → HIDDEN
layers.Dense(1, activation = 'sigmoid') → OUTPUT
)

model.compile(optimizer = 'sgd', loss = 'binary_crossentropy', metrics = 'accuracy')

history = model.fit(x_train.values, y_train.values,
validation_data = (x_test.values, y_test.values),
epochs = 100, batch_size = 4, verbose = 0)

VALIDATION LOSS

val_loss

This is the validation loss.

After generalizing well or just memorizing set (the split you gave with `validation_data = ...`) and calculates the loss on that unseen data.

This tells you whether your model is generalizing well or just memorizing the training data.

LOSS FUNCTIONS

Regression



→ MSE (Mean Squared Error)

→ MAE (Mean Absolute Error)

→ Huber loss

→ MSLE (we just use log over the errors).

classification

→ Binary cross entropy

→ classification cross entropy

$$MSE = \frac{1}{2} (\hat{y} - y)^2$$

$$\text{Cost function} = \frac{1}{2} \sum_{i=1}^n (\hat{y} - y)^2$$

PROs

- Very common
- Penalizes the big error.

CONs

- sensitive to outliers.

HUBER LOSS

- If we are having small errors, we will use MSE
- If we are having large errors, we will use MAE

LOSS FUNCTION FOR CLASSIFICATION

ANIMAL

0

1

1

0

⋮

⋮

⋮



BINARY CROSS
ENTROPY

ANIMAL

cat

dog

horse

cat

-

⋮

⋮

⋮



CATEGORICAL
CROSS
ENTROPY

ANIMAL

0

1

3

2

2

4

5

⋮



SOFT MAX
SPARSE CATEGORICAL
ENTROPY

OPTIMIZERS

SGD $\rightarrow$ Stochastic Gradient Descent

- You send one row at a time.
- Compute cost for that single row.
- Update weights immediately.
- With 1 lakh rows, in 1 epoch $\rightarrow$ 1 lakh updates.
- In 100 epochs $\rightarrow$ 1 crore updates.

ADAGRAD

Adaptive Gradient Descent.

$$w_T = w_{T-1} - \eta \frac{dL}{dw_{T-1}}$$

0.01
constant

$$w_T = w_{T-1} - \eta' \frac{dL}{dw_{T-1}}$$

$$\eta' = \frac{\eta}{\sqrt{\alpha_T + \epsilon}}$$

epsilon

$$\alpha_T = \sum_{i=1}^T \left(\frac{dL}{dw_i} \right)^2 \Rightarrow$$

ADAM OPTIMIZER

↳ Adagrad momentum.

$$w_i = w_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}$$

